

Login und Adresserfassung Schritt für Schritt

Schulungsunterlage zur Adressverwaltung v2.3.0 · Technische Dokumentation zum Sequenzdiagramm · Stand 4. Oktober 2026

1. Ziel dieser Unterlage

Diese Unterlage erklärt jeden der 38 nummerierten Schritte im Sequenzdiagramm der Adressverwaltung (documentation/Sequenzdiagramm.svg). Das Diagramm zeigt zwei Abläufe: die Anmeldung (Schritte 1 bis 16) und das Erfassen einer neuen Adresse (Schritte 17 bis 38).

Nach dem Durcharbeiten können Sie erklären, welchen Weg eine Anfrage vom Browser bis in die Datenbank nimmt, welche Rolle die beiden Tokens spielen und an welcher Stelle im Code jeder Schritt umgesetzt ist.

Zu jedem Schritt finden Sie drei Angaben: **Was passiert** beschreibt den Vorgang, **Im Code** nennt die Datei und die Stelle, **Warum so** begründet die Lösung oder weist auf eine Stolperfalle hin. Pfade im Frontend beziehen sich auf frontend/, Pfade im Backend auf backend/AdressverwaltungApi/.

2. So lesen Sie das Diagramm

Jede senkrechte Linie steht für einen Beteiligten, die Zeit läuft von oben nach unten. Ein durchgezogener Pfeil ist eine Anfrage, ein gestrichelter Pfeil eine Antwort. Ein Pfeil, der zum selben Beteiligten zurückführt, ist ein interner Arbeitsschritt. Ein Rahmen mit «alt» enthält zwei Alternativen, von denen genau eine abläuft; ein Rahmen mit «opt» läuft nur unter der genannten Bedingung ab.

3. Die Beteiligten

Beteiligter	Aufgabe	Im Projekt
Benutzer	Die Person vor dem Bildschirm.	–
Browser (Client Components)	Führt den React-Code der Seiten aus, zeigt Formulare an und sendet Anfragen.	frontend/app, frontend/components, frontend/lib/api.ts
nginx :443	Einziger Zugang von aussen. Beendet TLS und verteilt Anfragen nach Pfad an Next.js oder das Backend.	nginx/nginx.conf
Next.js / NextAuth	Liefert die Seiten aus und verwaltet die Sitzung des Frontends unter /api/auth/*.	frontend/lib/auth.ts, frontend/middleware.ts
Backend (ASP.NET Core :8080)	Die API: prüft Anmeldedaten, stellt Tokens aus, speichert und liefert Daten über OData.	backend/AdressverwaltungApi
PostgreSQL	Speichert Benutzer, Adressen, Städte und Einstellungen.	Tabellen Users, Adressen, Cities, Settings
SMTP-Server	Externer Mailserver für die Benachrichtigung bei neuen Adressen.	Konfiguration SmtSettings

4. Zwei Grundideen vorab

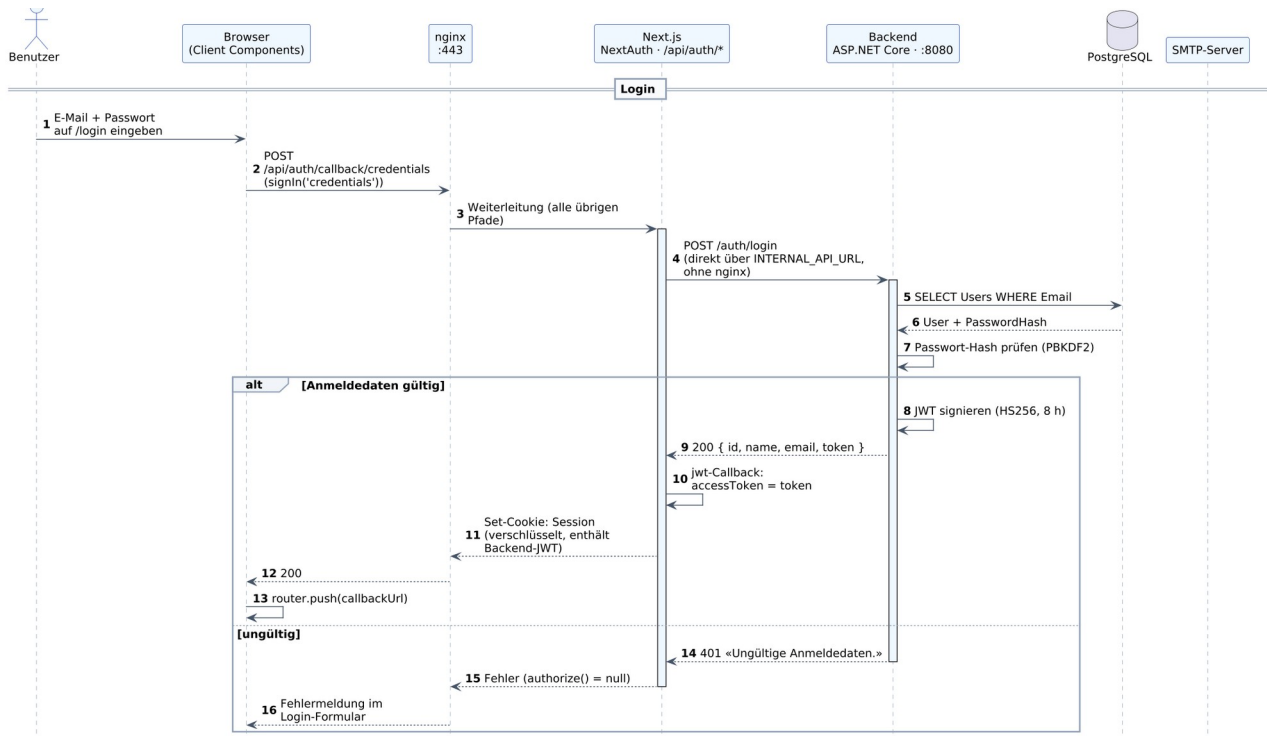
Ein Eingang, zwei Ziele. Der Browser spricht ausschliesslich mit nginx unter https://localhost. nginx leitet

/odata/*, /auth/* und /settings an das Backend und alles andere an Next.js. Backend und Datenbank sind von aussen nicht direkt erreichbar.

Zwei Tokens. Das Backend stellt beim Login ein JWT aus. NextAuth verpackt dieses JWT in seine eigene, verschlüsselte Sitzung und legt sie als Cookie im Browser ab. Für jeden API-Aufruf holt der Browser das Backend-JWT aus der Sitzung und sendet es im Authorization-Header mit. Beide gelten 8 Stunden.

5. Ablauf A: Login (Schritte 1 bis 16)

Die Schritte 1 bis 7 laufen immer ab. Danach verzweigt der Ablauf: Bei gültigen Anmeldedaten folgen die Schritte 8 bis 13, bei ungültigen die Schritte 14 bis 16.



Schritt 1: E-Mail und Passwort auf /login eingeben

Beteiligte: Benutzer → Browser

Was passiert: Die Benutzerin öffnet die Anwendung. Weil sie noch keine Sitzung hat, leitet der Seitenschutz sie auf die Login-Seite um. Dort gibt sie E-Mail-Adresse und Passwort ein und klickt auf «Anmelden». Bis hierhin verlässt nichts den Browser: Die Eingaben stehen nur im Zustand der React-Komponente.

Im Code: Der Seitenschutz ist `frontend/middleware.ts`: Jede Seite ausser `/login` und `/api/auth` verlangt eine gültige Sitzung. Das Formular ist `app/login/LoginForm.tsx`; die Felder sind Controlled Inputs (`useState`).

Warum so: Das Formular ist eine Client-Komponente, weil es auf Eingaben reagieren muss. Die umgebende Seite `app/login/page.tsx` bleibt eine Server-Komponente und entscheidet dort, ob die Google-Schaltfläche angezeigt wird.

Schritt 2: POST /api/auth/callback/credentials

Beteiligte: Browser → nginx

Was passiert: Beim Absenden ruft das Formular die NextAuth-Funktion `signIn('credentials')` auf. Sie holt zuerst ein CSRF-Token und sendet dann E-Mail und Passwort per POST an den NextAuth-Endpunkt. Die Verbindung ist mit TLS verschlüsselt und endet bei nginx auf Port 443.

Im Code: `app/login/LoginForm.tsx`, Funktion `handleCredentials: signIn('credentials', { email,`

`password, redirect: false, callbackUrl })).redirect: false` bewirkt, dass NextAuth nicht selbst weiterleitet, sondern das Ergebnis an das Formular zurückgibt.

Warum so: Das Passwort geht nie direkt vom Browser an das Backend, sondern immer über NextAuth. nginx begrenzt diesen Pfad auf 10 Anfragen pro Minute und Client-IP; wer mehr sendet, erhält die Antwort 429. Das bremst das Durchprobieren von Passwörtern.

Schritt 3: Weiterleitung an Next.js

Beteiligte: nginx → Next.js

Was passiert: nginx entscheidet anhand des Pfads, wohin eine Anfrage gehört. `/odata/`, `/auth/` und `/settings` gehen an das Backend, alles andere an Next.js. `/api/auth/callback/credentials` fällt unter «alles andere» und landet beim Next.js-Server auf Port 3000.

Im Code: `nginx/nginx.conf`: der Block `location /api/auth/callback/credentials` (mit Ratenbegrenzung) und der allgemeine Block `location /`.

Warum so: Achtung Verwechslungsgefahr: `/api/auth` gehört zu NextAuth im Frontend, `/auth/` zum Backend. Deshalb heisst die Login-Seite `/login` und die Einstellungsseite `/einstellungen` – Seiten dürfen nicht mit den Backend-Präfixen kollidieren.

Schritt 4: POST /auth/login (intern, ohne nginx)

Beteiligte: Next.js → Backend

Was passiert: NextAuth ruft für den Credentials-Provider die Funktion `authorize()` auf. Diese läuft auf dem Next.js-Server und sendet E-Mail und Passwort als JSON an den Login-Endpunkt des Backends. Der Aufruf geht direkt über das Docker-Netzwerk an `http://backend:8080`, nicht über nginx.

Im Code: `frontend/lib/auth.ts`, `authorize()` im `CredentialsProvider`. Die Adresse stammt aus `INTERNAL_API_URL`. Im Backend nimmt `AuthController.Login` die Anfrage entgegen; der Body wird auf `LoginRequest` abgebildet und geprüft (E-Mail-Format, Längen).

Warum so: Server-seitige Aufrufe brauchen kein TLS und keinen Umweg über nginx, weil beide Container im selben internen Netz liegen. Das Backend ist von aussen nur über nginx erreichbar.

Schritt 5: SELECT Users WHERE Email

Beteiligte: Backend → PostgreSQL

Was passiert: Das Backend sucht den Benutzer anhand der E-Mail-Adresse. Die Suche ignoriert Gross- und Kleinschreibung, indem beide Seiten in Grossbuchstaben verglichen werden.

Im Code: `Controllers/AuthController.cs`, Methode `FindUserByEmailAsync`. Entity Framework Core übersetzt den LINQ-Ausdruck in ein parametrisiertes SQL-SELECT auf die Tabelle `users`.

Warum so: Die E-Mail wird nie in den SQL-Text eingesetzt, sondern als Parameter übergeben. SQL-Injection ist damit an dieser Stelle ausgeschlossen.

Schritt 6: User und PasswordHash

Beteiligte: PostgreSQL → Backend

Was passiert: Die Datenbank liefert den Datensatz des Benutzers zurück – oder nichts, wenn die E-Mail unbekannt ist. Im Datensatz steht nicht das Passwort, sondern nur dessen Hash.

Im Code: Modell `Models/User.cs` mit den Feldern `Email`, `PasswordHash`, `DisplayName`. Der Hash enthält Version, Verfahren, Anzahl Iterationen und Salt, sodass er sich später wieder prüfen lässt.

Warum so: Ein Hash lässt sich nicht zurückrechnen. Selbst wer die Datenbank liest, kennt die Passwörter nicht.

Schritt 7: Passwort-Hash prüfen (PBKDF2)

Beteiligte: Backend (interner Schritt)

Was passiert: Das Backend berechnet aus dem eingegebenen Passwort und dem gespeicherten Salt erneut den Hash und vergleicht ihn mit dem gespeicherten Wert. PBKDF2 wiederholt die Berechnung viele tausend Mal und ist dadurch absichtlich langsam.

Im Code: `Controllers/AuthController.cs: VerifyHashedPassword` aus `IPasswordHasher<User>`. Ist der Benutzer unbekannt, wird gegen einen festen Ersatz-Hash (DummyHash) geprüft.

Warum so: Die Prüfung läuft auch bei unbekannter E-Mail. So dauert die Antwort in beiden Fällen gleich lange, und ein Angreifer kann aus der Antwortzeit nicht ablesen, welche E-Mail-Adressen registriert sind.

Schritt 8: JWT signieren (HS256, 8 h) – Fall «gültig»

Beteiligte: Backend (interner Schritt)

Was passiert: Stimmen E-Mail und Passwort, stellt das Backend ein JSON Web Token aus. Es enthält die Benutzer-Id (sub), E-Mail, Anzeigenamen, eine eindeutige Token-Id und das Ablaufdatum in 8 Stunden. Das Token wird mit einem geheimen Schlüssel signiert.

Im Code: `Services/JwtTokenService.cs, Methode CreateToken`. Schlüssel, Aussteller, Empfänger und Laufzeit kommen aus `JwtOptions` (Abschnitt `Jwt` der Konfiguration, im Docker-Stack aus `.env`).

Warum so: Das Token ist signiert, nicht verschlüsselt: Jeder kann den Inhalt lesen, aber niemand kann ihn ohne den Schlüssel verändern. Deshalb muss der Schlüssel geheim bleiben, und es gehören keine vertraulichen Daten ins Token.

Schritt 9: 200 { id, name, email, token }

Beteiligte: Backend → Next.js

Was passiert: Das Backend antwortet mit Status 200 und einem JSON-Objekt, das die Benutzerdaten und das eben ausgestellte Token enthält.

Im Code: `LoginResponse` in `Dtos/AuthDtos.cs`. Der Passwort-Hash ist bewusst kein Teil dieses Typs.

Warum so: Das Format `{ id, name, email }` ist das, was NextAuth von `authorize()` erwartet. Das zusätzliche Feld `token` transportiert das Backend-JWT.

Schritt 10: jwt-Callback: accessToken = token

Beteiligte: Next.js (interner Schritt)

Was passiert: NextAuth baut nun seine eigene Sitzung. Im `jwt-Callback` übernimmt das Frontend die Benutzer-Id und das Backend-JWT in das NextAuth-Token.

Im Code: `frontend/lib/auth.ts, callbacks.jwt`: Beim Login ist `user` gesetzt, also werden `token.id` und `token.accessToken` geschrieben. Die Typen dafür stehen in `types/next-auth.d.ts`.

Warum so: Hier treffen sich die zwei Token-Schichten: Das NextAuth-Token ist die Sitzung des Frontends, das Backend-JWT darin ist der Ausweis gegenüber der API.

Schritt 11: Set-Cookie: Session (verschlüsselt)

Beteiligte: Next.js → nginx

Was passiert: NextAuth verschlüsselt sein Token mit `NEXTAUTH_SECRET` und schickt es als Cookie zurück. Das Cookie ist `HttpOnly`, also für JavaScript im Browser nicht lesbar, und gilt 8 Stunden.

Im Code: `frontend/lib/auth.ts: session: { strategy: 'jwt', maxAge: 8 * 60 * 60 }`. Die Verschlüsselung übernimmt NextAuth selbst.

Warum so: Sitzungsdauer im Frontend und Laufzeit des Backend-JWT betragen beide 8 Stunden und müssen gemeinsam geändert werden. Sonst gäbe es eine Sitzung, deren Backend-Token bereits abgelaufen ist.

Schritt 12: 200

Beteiligte: nginx → Browser

Was passiert: nginx reicht die Antwort samt Cookie an den Browser weiter. Der Browser speichert das Cookie und sendet es von nun an bei jeder Anfrage an dieselbe Adresse automatisch mit.

Im Code: nginx/nginx.conf ergänzt die Sicherheits-Header (unter anderem Strict-Transport-Security und Content-Security-Policy).

Warum so: Weil alles über dieselbe Adresse (https://localhost) läuft, braucht es keine CORS-Freigaben für den Browser.

Schritt 13: router.push(callbackUrl)

Beteiligte: Browser (interner Schritt)

Was passiert: Das Formular erhält das Ergebnis von `signIn('credentials')` ohne Fehler und navigiert zur Zielseite – normalerweise zur Adressliste, oder zu der Seite, die ursprünglich aufgerufen wurde.

Im Code: app/login/LoginForm.tsx: `router.push(callbackUrl)` und `router.refresh()`.
`sichereCallbackUrl()` lässt nur Pfade innerhalb der eigenen App zu.

Warum so: Ohne diese Prüfung könnte ein präparierter Link nach der Anmeldung auf eine fremde Seite weiterleiten (Open Redirect).

Schritt 14: 401 «Ungültige Anmeldedaten.» – Fall «ungültig»

Beteiligte: Backend → Next.js

Was passiert: Ist die E-Mail unbekannt oder das Passwort falsch, antwortet das Backend mit Status 401 und einer bewusst allgemeinen Meldung. Es wird kein Token ausgestellt.

Im Code: controllers/AuthController.cs: `return Unauthorized(new { message = "Ungültige Anmeldedaten." })`.

Warum so: Die Meldung verrät nicht, ob die E-Mail oder das Passwort falsch war. Das erschwert das Ausspähen gültiger Konten.

Schritt 15: Fehler (authorize() = null)

Beteiligte: Next.js → nginx

Was passiert: `authorize()` sieht, dass die Antwort nicht erfolgreich war, und gibt `null` zurück. Für NextAuth heisst das: Anmeldung abgelehnt. Es wird kein Sitzungs-Cookie gesetzt.

Im Code: frontend/lib/auth.ts: `if (!response.ok) return null`. Auch Netzwerk- und Parse-Fehler führen über den `catch`-Zweig zu `null`.

Warum so: Im Zweifel wird die Anmeldung verweigert. Ein nicht erreichbares Backend führt nie zu einer gültigen Sitzung.

Schritt 16: Fehlermeldung im Login-Formular

Beteiligte: nginx → Browser

Was passiert: Der Browser erhält das Ergebnis mit gesetztem Fehler. Das Formular bleibt auf der Login-Seite und zeigt «Ungültige E-Mail-Adresse oder falsches Passwort.» an.

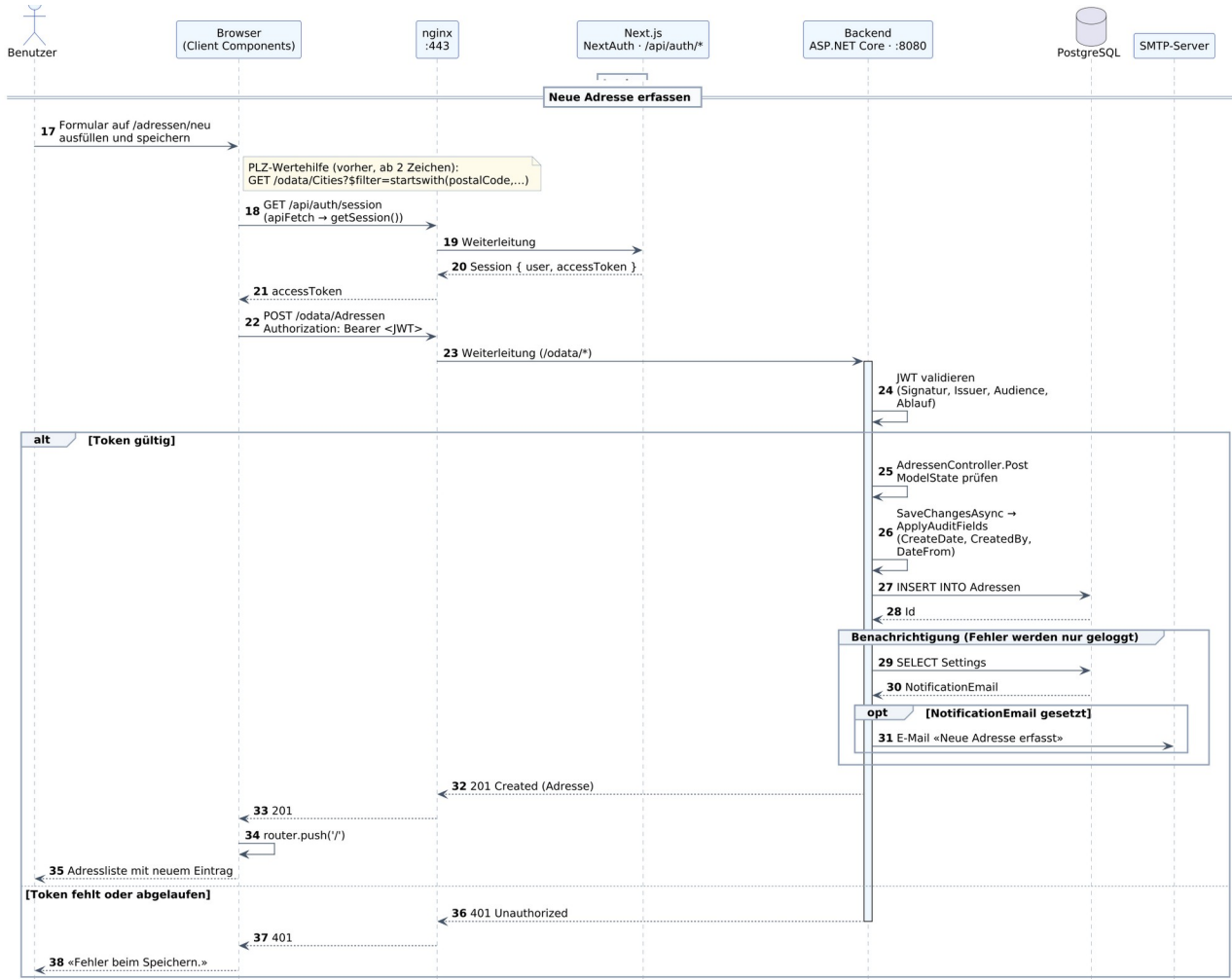
Im Code: app/login/LoginForm.tsx: `result?.error` setzt den Zustand `fehler`; die Komponente

Fehlermeldung zeigt ihn an.

Warum so: Die Meldung erscheint im Formular statt in einem Browser-Dialog. So bleibt die Eingabe erhalten und die Benutzerin kann es direkt erneut versuchen.

6. Ablauf B: Neue Adresse erfassen (Schritte 17 bis 38)

Die Schritte 17 bis 24 laufen immer ab. Ist das Token gültig, folgen die Schritte 25 bis 35; fehlt es oder ist es abgelaufen, die Schritte 36 bis 38.



Schritt 17: Formular auf /adressen/neu ausfüllen und speichern

Beteiligte: Benutzer → Browser

Was passiert: Die angemeldete Benutzerin öffnet «+ Neue Adresse», füllt Vorname, Name, Strasse, Nummer, PLZ und Ort aus und klickt auf «Adresse speichern». Schon beim Tippen der PLZ hilft die Wertehilfe (gelbe Notiz im Diagramm): Ab zwei Zeichen fragt der Browser passende Städte ab und bietet sie zur Auswahl an; die Auswahl füllt PLZ und Ort.

Im Code: app/adressen/neu/page.tsx mit components/AdresseForm.tsx. Wertehilfe: sucheStaedteNachPlz in frontend/lib/api.ts, nach 300 ms Eingabepause, höchstens 10 Treffer (\$filter=startswith(postalCode, '80')&\$top=10).

Warum so: Die Verzögerung (Debounce) verhindert, dass bei jedem Tastendruck eine Anfrage ausgelöst wird. Die Wertehilfe ist eine Hilfe, keine Pflicht: PLZ und Ort lassen sich auch frei eingeben.

Schritt 18: GET /api/auth/session

Beteiligte: Browser → nginx

Was passiert: Bevor die Adresse gesendet wird, braucht der Browser das Backend-JWT. Dieses steckt im verschlüsselten Sitzungs-Cookie, das JavaScript nicht lesen kann. Deshalb fragt die Hilfsfunktion `apiFetch` die Sitzung bei NextAuth ab; das Cookie geht dabei automatisch mit.

Im Code: `frontend/lib/api.ts`: `apiFetch` ruft im Browser `getSession()` aus `next-auth/react` auf. Auf dem Server (Server-Komponenten) verwendet dieselbe Funktion stattdessen `getServerSession(authOptions)`.

Warum so: Alle Backend-Aufrufe laufen über `apiFetch`. So ist an einer einzigen Stelle geregelt, wie das Token beschafft und angehängt wird.

Schritt 19: Weiterleitung

Beteiligte: nginx → Next.js

Was passiert: Auch `/api/auth/session` beginnt nicht mit einem Backend-Präfix. nginx leitet die Anfrage an Next.js weiter.

Im Code: `nginx/nginx.conf`, Block `location /`.

Warum so: Gleiche Regel wie in Schritt 3.

Schritt 20: Session { user, accessToken }

Beteiligte: Next.js → nginx

Was passiert: NextAuth entschlüsselt das Cookie mit `NEXTAUTH_SECRET`, prüft das Ablaufdatum und baut daraus das Sitzungsobjekt. Der `session`-Callback fügt die Benutzer-Id und das Backend-JWT als `accessToken` hinzu.

Im Code: `frontend/lib/auth.ts`, `callbacks.session`: `session.accessToken = token.accessToken`.

Warum so: Fehlt das Cookie oder ist es abgelaufen, kommt ein leeres Objekt zurück. Dann gibt es kein Token, und der folgende Aufruf endet im Fehlerfall (Schritte 36 bis 38).

Schritt 21: accessToken

Beteiligte: nginx → Browser

Was passiert: Der Browser erhält das Sitzungsobjekt und entnimmt ihm das Backend-JWT.

Im Code: `frontend/lib/api.ts`: `token = session?.accessToken`.

Warum so: Ab hier kennt JavaScript im Browser das Backend-JWT. Das ist ein bewusster Kompromiss dieser Architektur: Der Browser spricht direkt mit der API. Eine XSS-Lücke könnte das Token auslesen – deshalb darf die App keine ungeprüften Inhalte als HTML einfügen.

Schritt 22: POST /odata/Adressen mit Authorization: Bearer

Beteiligte: Browser → nginx

Was passiert: Jetzt sendet der Browser die Adresse als JSON an die API. Das Token steht im Header `Authorization: Bearer <JWT>`. Die URL ist relativ (`/odata/Adressen`), geht also an dieselbe Adresse wie die Seite selbst.

Im Code: `frontend/lib/api.ts`: `erstelleAdresse` → `apiRequest` mit `jsonRequest('POST', adresse)`. Gesendet wird ein `AdresseCreate`: die sechs Fachfelder in camelCase, ohne Id und ohne Audit-Felder.

Warum so: Im Browser ist die Basis-URL leer; nginx sorgt für die Weiterleitung. Darum funktionieren die Formulare nur über den nginx-Stack und nicht mit `npm run dev` allein.

Schritt 23: Weiterleitung (/odata/*)

Beteiligte: nginx → Backend

Was passiert: Der Pfad beginnt mit /odata/. nginx leitet die Anfrage an das Backend (http://backend:8080) weiter und reicht den Authorization: Bearer <JWT>-Header unverändert durch.

Im Code: nginx/nginx.conf, Block location /odata/.

Warum so: Eine neue Backend-Route mit eigenem Präfix braucht einen eigenen location-Block. Fehlt er, landet die Anfrage bei Next.js und endet mit 404.

Schritt 24: JWT validieren

Beteiligte: Backend (interner Schritt)

Was passiert: Bevor ein Controller läuft, prüft die Authentifizierungs-Middleware das Token: Stimmt die Signatur mit dem geheimen Schlüssel? Stimmen Aussteller und Empfänger? Ist das Token noch nicht abgelaufen? Aus einem gültigen Token entsteht die Identität des Aufrufers; der Claim name wird zum Benutzernamen.

Im Code: Program.cs: AddJwtBearer mit TokenValidationParameters, danach UseAuthentication() und UseAuthorization(). [Authorize] auf ODataCrudController<TEntity> verlangt die Anmeldung für alle OData-Endpunkte.

Warum so: Das Backend fragt dafür weder die Datenbank noch das Frontend. Es vertraut dem Token, weil nur es selbst den Signaturschlüssel kennt. Das macht die Prüfung schnell, bedeutet aber auch: Ein ausgestelltes Token gilt bis zum Ablauf.

Schritt 25: AdressenController.Post – ModelState prüfen (Fall «Token gültig»)

Beteiligte: Backend (interner Schritt)

Was passiert: OData bildet den JSON-Body auf ein Adresse-Objekt ab und ruft die Post-Methode auf. Diese prüft zuerst, ob die Daten gültig sind: Alle sechs Felder sind Pflicht und haben eine Höchstlänge. Bei Verstößen antwortet das Backend mit 400 und nennt die betroffenen Felder.

Im Code: Post ist in der Basisklasse ODataCrudController<TEntity> implementiert; AdressenController erbt sie. Die Regeln stehen als [Required] und [MaxLength] am Modell Models/Adresse.cs.

Warum so: Der Browser prüft die Pflichtfelder zwar auch, aber nur das Backend ist verbindlich: Eine Anfrage kann jederzeit am Formular vorbei gesendet werden.

Schritt 26: SaveChangesAsync → ApplyAuditFields

Beteiligte: Backend (interner Schritt)

Was passiert: Vor dem Speichern füllt der Datenbankkontext die Audit-Felder: CreateDate mit dem aktuellen Zeitpunkt, CreatedBy mit dem Namen aus dem Token und DateFrom mit dem heutigen Datum, falls nichts übergeben wurde.

Im Code: Data/AdresseDbContext.cs: SaveChangesAsync ruft ApplyAuditFields auf. Der Benutzername kommt aus HttpContext.User.Identity.Name; ohne angemeldeten Benutzer (zum Beispiel beim Seeding) steht dort "system".

Warum so: Die Felder werden an einer zentralen Stelle gesetzt und gelten für alle Entitäten. Controller setzen sie nie selbst, und Werte aus dem Request werden überschrieben – so lassen sie sich nicht fälschen.

Schritt 27: INSERT INTO Adressen

Beteiligte: Backend → PostgreSQL

Was passiert: Entity Framework Core erzeugt aus dem Objekt ein parametrisiertes INSERT und sendet es an

PostgreSQL.

Im Code: `Data/AdresseDbContext.cs: DbSet<Adresse> Adressen; Tabelle Adressen.` Zeitstempel werden als `timestamp without time zone` gespeichert.

Warum so: Deshalb setzt `ApplyAuditFields` die Zeit mit `DateTimeKind.Unspecified`: Npgsql lehnt UTC-Zeitwerte für diesen Spaltentyp ab.

Schritt 28: Id

Beteiligte: PostgreSQL → Backend

Was passiert: Die Datenbank vergibt die neue Id und gibt sie zurück. Entity Framework Core trägt sie in das Objekt ein. Damit ist die Adresse dauerhaft gespeichert.

Im Code: Die Id ist eine automatisch hochgezählte Spalte; der Client übergibt sie nicht.

Warum so: Alles, was jetzt noch folgt, kann das Speichern nicht mehr rückgängig machen – wichtig für die Benachrichtigung im nächsten Block.

Schritt 29: SELECT Settings (Block «Benachrichtigung»)

Beteiligte: Backend → PostgreSQL

Was passiert: Nach dem Speichern löst der Controller die Benachrichtigung aus. Der Dienst liest zuerst die Einstellungen, um die Empfängeradresse zu erfahren.

Im Code: `AdressenController: OnCreatedAsync` ruft `INotificationService` auf; Implementierung `EmailNotificationService`, Methode `NotifyNewAdresseAsync`.

Warum so: Der Controller kennt nur die Schnittstelle. Ob per E-Mail, SMS oder Webhook benachrichtigt wird, lässt sich ändern, ohne den Controller anzufassen.

Schritt 30: NotificationEmail

Beteiligte: PostgreSQL → Backend

Was passiert: Die Einstellungen bestehen aus einer einzigen Zeile mit der Benachrichtigungs-E-Mail. Sie wird auf der Seite «Einstellungen» gepflegt.

Im Code: `Modell Models/Settings.cs`; gepflegt über `SettingsController (GET/PUT /settings)`.

Warum so: Gibt es keine Zeile oder ist die Adresse leer, schreibt der Dienst nur einen Log-Eintrag und endet. Schritt 31 entfällt dann (Rahmen «opt» im Diagramm).

Schritt 31: E-Mail «Neue Adresse erfasst»

Beteiligte: Backend → SMTP-Server

Was passiert: Ist eine Adresse hinterlegt, sendet das Backend eine Text-E-Mail mit Name, Strasse und Ort der neuen Adresse an den SMTP-Server.

Im Code: `Services/EmailService.cs`, Methode `SendAsync`; Zugangsdaten aus `smtpOptions` (Abschnitt `SmtpSettings`).

Warum so: Der ganze Block steht in einem `try/catch`: Schlägt der Versand fehl, wird der Fehler geloggt, aber die Anfrage läuft normal weiter. Ein gestörter Mailserver darf das Erfassen von Adressen nicht verhindern.

Schritt 32: 201 Created (Adresse)

Beteiligte: Backend → nginx

Was passiert: Das Backend antwortet mit Status 201 und der gespeicherten Adresse als JSON – jetzt mit Id und Audit-Feldern.

Im Code: `odataCrudController<TEntity>: return Created(entity).`

Warum so: 201 bedeutet «neu angelegt» und ist genauer als ein allgemeines 200.

Schritt 33: 201

Beteiligte: nginx → Browser

Was passiert: nginx reicht die Antwort an den Browser weiter.

Im Code: `frontend/lib/api.ts: apiRequest prüft response.ok; erstelleAdresse wandelt die Antwort mit normalizeAdresse in den Typ Adresse um.`

Warum so: Die `normalize*`-Funktionen sorgen dafür, dass der Rest des Frontends mit einem festen Typ arbeitet, unabhängig von Details der API-Antwort.

Schritt 34: router.push('/')

Beteiligte: Browser (interner Schritt)

Was passiert: Die Seite navigiert zurück zur Adressliste.

Im Code: `app/adressen/neu/page.tsx: handleSubmit mit router.push('/')`.

Warum so: Die Navigation geschieht erst nach erfolgreichem Speichern. Bei einem Fehler bleibt das Formular mit den Eingaben stehen.

Schritt 35: Adressliste mit neuem Eintrag

Beteiligte: Browser → Benutzer

Was passiert: Die Adressliste lädt alle Adressen neu und zeigt den neuen Eintrag. Dieser Abruf läuft wieder nach dem Muster der Schritte 18 bis 24: Sitzung abfragen, dann GET `/odata/Adressen` mit Bearer-Token.

Im Code: `app/page.tsx: ladeDaten ruft getAlleAdressen()` auf.

Warum so: Die Liste wird vom Backend geholt und nicht lokal ergänzt. So zeigt sie immer den gespeicherten Stand.

Schritt 36: 401 Unauthorized – Fall «Token fehlt oder abgelaufen»

Beteiligte: Backend → nginx

Was passiert: Fehlt das Token, ist es abgelaufen oder ist die Signatur ungültig, bricht die Middleware aus Schritt 24 die Anfrage ab. Der Controller wird nicht aufgerufen, es wird nichts gespeichert.

Im Code: `[Authorize]` zusammen mit der `JwtBearer-Middleware` in `Program.cs`.

Warum so: Typischer Auslöser: Die Seite war länger als 8 Stunden offen, oder der Signaturschlüssel wurde seit der Anmeldung geändert.

Schritt 37: 401

Beteiligte: nginx → Browser

Was passiert: nginx gibt die Antwort unverändert weiter.

Im Code: –

Warum so: nginx bewertet Statuscodes des Backends nicht; es leitet nur weiter.

Schritt 38: «Fehler beim Speichern.»

Beteiligte: Browser → Benutzer

Was passiert: `apiRequest` erkennt den Fehlerstatus und wirft einen Fehler. Das Formular fängt ihn ab und zeigt «Fehler beim Speichern. Bitte versuche es erneut.» an. Die Eingaben bleiben erhalten.

Im Code: `frontend/lib/api.ts: throw new Error(...); components/AdresseForm.tsx: catch-Zweig in handleSubmit`, Anzeige über Fehlermeldung.

Warum so: Die Meldung ist allgemein gehalten. Bei einem abgelaufenen Token hilft erneutes Versuchen nicht – die Benutzerin muss sich neu anmelden.

7. Das Wichtigste in Kürze

- Der Browser kennt nur eine Adresse. `nginx` entscheidet nach dem Pfad, ob `Next.js` oder das Backend antwortet.
- Das Passwort wird nur einmal übertragen, beim Login. Danach weist sich der Browser mit dem Token aus.
- Das Backend vertraut keinem Wert aus dem Request, der sich aus dem Token ergibt: Den Benutzernamen für die Audit-Felder nimmt es aus dem Token.
- Validierung im Browser ist Komfort, Validierung im Backend ist verbindlich.
- Die Benachrichtigung ist ein Nebeneffekt. Ihr Scheitern ändert nichts am Ergebnis der Anfrage.
- Sitzung und Backend-JWT laufen gemeinsam nach 8 Stunden ab.

8. Kontrollfragen

Frage	Antwort
1. Warum sendet der Browser das Passwort nicht direkt an <code>/auth/login</code> des Backends?	Die Anmeldung läuft über <code>NextAuth</code> , damit das Frontend eine eigene Sitzung (verschlüsseltes Cookie) aufbauen kann. Den Aufruf an das Backend macht der <code>Next.js</code> -Server intern (Schritte 2 bis 4).
2. Welche zwei Tokens gibt es, und wer prüft welches?	Das Sitzungs-Token von <code>NextAuth</code> im Cookie prüft <code>Next.js</code> (Schritt 20). Das Backend-JWT im <code>Authorization-Header</code> prüft das Backend (Schritt 24).
3. Woher weiss das Backend, wer eine Adresse erfasst hat?	Aus dem Claim <code>name</code> des geprüften Tokens; <code>ApplyAuditFields</code> schreibt ihn nach <code>CreatedBy</code> (Schritte 24 und 26).
4. Was geschieht mit der neuen Adresse, wenn der Mailserver nicht erreichbar ist?	Sie bleibt gespeichert und die Antwort ist 201. Der Fehler wird nur geloggt (Schritte 28 bis 32).
5. Warum braucht eine neue Backend-Route unter <code>/berichte</code> eine Änderung in <code>nginx</code> ?	Weil <code>nginx</code> nur <code>/odata/</code> , <code>/auth/</code> und <code>/settings</code> an das Backend leitet. Alles andere geht an <code>Next.js</code> (Schritte 3 und 23).
6. Die Benutzerin lässt das Formular über Nacht offen und speichert am Morgen. Was passiert?	Sitzung und Backend-JWT sind nach 8 Stunden abgelaufen. Der Aufruf endet mit 401, das Formular zeigt «Fehler beim Speichern.» (Schritte 36 bis 38). Sie muss sich neu anmelden.

9. Glossar

Begriff	Bedeutung
JWT	JSON Web Token: signiertes Datenpaket mit Angaben zum Benutzer und einem Ablaufdatum.
Bearer-Token	Token, das im Header <code>Authorization: Bearer ...</code> mitgesendet wird. Wer es besitzt, gilt als

	angemeldet.
HS256	Signaturverfahren mit einem gemeinsamen geheimen Schlüssel (HMAC-SHA256).
PBKDF2	Verfahren, das aus Passwort und Salt in vielen Wiederholungen einen Hash berechnet.
Salt	Zufallswert pro Passwort, damit gleiche Passwörter verschiedene Hashes ergeben.
NextAuth	Bibliothek für Anmeldung und Sitzungen in Next.js.
OData	Standard für REST-Schnittstellen mit einheitlichen Abfrageoptionen wie <code>\$filter</code> und <code>\$top</code> .
Reverse Proxy	Vorgeschalteter Server, der Anfragen entgegennimmt und an interne Dienste verteilt (hier nginx).
Claim	Einzelne Angabe im Token, zum Beispiel <code>name</code> oder <code>email</code> .
Audit-Felder	Felder, die festhalten, wer einen Datensatz wann erstellt und geändert hat.
Controlled Input	Formularfeld, dessen Wert im React-Zustand liegt.